

ASSIGNMENT-2

QUERY QUEST

NAME- NIKHIL BENIWAL

ROLL NO-240693

PART-1

Question-1) Consider a student table where the phone_number column allows NULL. A classmate writes the following query to count students without a phone number:

SELECT COUNT (*) FROM Student WHERE phone_number = NULL;

This query always returns 0, even when such students exist. Explain why and rewrite the query correctly. Then explain, in general terms, why NULL is excluded from most aggregate computations and comparison operations.

ANS: We cannot use comparison operators with NULL because in SQL NULL doesn't mean 0 it means something missing or unknown;

Correct syntax: **SELECT count (*) FROM Student WHERE phone_number is NULL;**

Question-2) A query uses SELECT DISTINCT department, COUNT (*) FROM Employee GROUP BY department; A student argues the DISTINCT keyword here is redundant. Is the student correct? Justify your answer by explaining what DISTINCT would be doing in this specific query.

ANS: Yes, The student is correct because here the GROUP BY will execute first and it will look at the table EMPLOYEE and combine the people within a department and write the number so now the table does not have any duplicate department so when DISTINCT will execute it will not change anything because we already have distinct departments.

Question-3) You run two queries on the same Customer and Order tables, one using INNER JOIN and one using LEFT JOIN (Customer as the left table) and get a different row count. What does this difference in row count tell you about the data, without looking at the tables directly? Describe the scenario that would cause the LEFT JOIN to return more rows.

ANS: When we use left join in tables then It will produce an output table which contains all the existing rows of left table and the values that are common in both the tables but in INNER JOIN we only take the rows whose values are common in both the tables so that's why we will get more rows when we perform left join because in inner join the values that are present in left table but not in right table are eliminated.

Question-4) Explain why the following query is invalid in standard SQL, and correct it: SELECT department, employee name, AVG (salary) FROM Employee GROUP BY department.

In your explanation, address what it would conceptually mean for SQL to "allow" this query to run as-is i.e., what value would it even display for employee name?

ANS: In this query, we used GROUP BY, which groups all employees from the same department into a single row. Because of this, it is not possible to print the individual names of all the people in that department. Instead, we can only ask for summary values using aggregate functions like AVG () or MAX ().

Correct: **SELECT department, AVG (salary) FROM Employee GROUP BY department;**

It will return a table in which the department and the average salary of all employees within that department will be written.

Question-5) A manager wants the list of departments where the average salary exceeds ₹80,000. A student writes:

SELECT department, AVG (salary) FROM Employee WHERE AVG (salary) > 80000 GROUP BY department; This fails. Identify the error, explain why WHERE cannot be used here (in terms of query execution order), and provide the corrected query.

ANS: In this query, WHERE cannot be used because when filtering grouped data, we must use HAVING instead of WHERE. The execution order in the original query is wrong; the database must group the rows first using GROUP BY, and only after that can we apply the condition using HAVING.

Correct: **SELECT department, AVG (salary) FROM Employee GROUP BY department having AVG (salary)>80000.**

Question-6) For each of the following, state whether a correlated or non-correlated subquery is more appropriate, and briefly justify:

(a) Find all products priced above the overall average product price.

(b) Find all employees who earn more than the average salary of their own department.

What is the key performance implication of a correlated subquery compared to a non-correlated one when the outer table is large?

ANS:

- (a) Non-correlated subquery: because overall average product price is a single and constant value and does not depend on outer query. The non correlated subquery is appropriate because it needs to be executed only once.
- (b) Correlated subquery: Because average salary changes depending on which department you are evaluating. To calculate the average salary for specific department you will need a reference to the department.

PART-2

(a) SELECT Customer.customer_id from Customer

Left join

`Order`

On Customer.customer_id=`Order`.customer_id

Where `Order`.customer_id is NULL

(b) WITH OrderTotals AS (

SELECT

`Order`.restaurant_id,

`Order`.order_id,

SUM (MenuItem.price * OrderItem.quantity) AS order_total

FROM `Order`

JOIN OrderItem

ON `Order`.order_id = OrderItem.order_id

JOIN MenuItem

ON OrderItem.item_id = MenuItem.item_id

```

GROUP BY

    `Order`.restaurant_id,

    `Order`.order_id
)

SELECT

    OrderTotals.restaurant_id,

    AVG(OrderTotals.order_total) AS avg_order_value

FROM OrderTotals

GROUP BY OrderTotals.restaurant_id

HAVING COUNT(OrderTotals.order_id) > 5;

(C)

WITH OrderTotals AS (

    SELECT

        `Order`.restaurant_id,

        `Order`.order_id,

        SUM(MenuItem.price * OrderItem.quantity) AS order_total

    FROM `Order`

    JOIN OrderItem

        ON `Order`.order_id = OrderItem.order_id

    JOIN MenuItem

        ON OrderItem.item_id = MenuItem.item_id

    GROUP BY

        `Order`.restaurant_id,

        `Order`.order_id

),

```

RestaurantAverages AS (

SELECT

OrderTotals.restaurant_id,

AVG(OrderTotals.order_total) AS avg_order_value

FROM OrderTotals

GROUP BY OrderTotals.restaurant_id

HAVING COUNT(OrderTotals.order_id) > 5

)

SELECT

RestaurantAverages.restaurant_id,

RestaurantAverages.avg_order_value

FROM RestaurantAverages

WHERE RestaurantAverages.avg_order_value > (

SELECT AVG(OrderTotals.order_total)

FROM OrderTotals

);

(d)

WITH ItemTotals AS (

SELECT

MenuItem.restaurant_id,

MenuItem.item_id,

MenuItem.item_name,

SUM (OrderItem.quantity) AS total_sold

FROM MenuItem

JOIN OrderItem

ON MenuItem.item_id = OrderItem.item_id

GROUP BY

MenuItem.restaurant_id,

MenuItem.item_id,

MenuItem.item_name

),

RankedItems AS (

SELECT

ItemTotals.restaurant_id,

ItemTotals.item_id,

ItemTotals.item_name,

ItemTotals.total_sold,

RANK() OVER(

PARTITION BY ItemTotals.restaurant_id

ORDER BY ItemTotals.total_sold DESC

) AS rank_num

FROM ItemTotals

)

SELECT

RankedItems.restaurant_id,

RankedItems.item_id,

RankedItems.item_name,

RankedItems.total_sold

FROM RankedItems

WHERE RankedItems.rank_num = 1;

PART-III

- (1) Primary key = (student_id, course_id)

Functional dependencies:

student_id -> student_id

course_id -> course_name, instructor_id

instructor_id -> instructor_name, instructor_office

student_id, course_id -> grade, enrollment_date

- (2) Yes, it is 1NF: All values holds single, atomic values (There are no multivalued attributes)
- (3) Attributes like student_name only depend on student_id. Attributes like course_name only depend on course_id. They do not depend on the *entire* composite primary key (student_id, course_id).

Anomaly Example (Insertion): You cannot add a brand-new course to the database until a student enrolls in it, because student_id is part of the primary key and cannot be left blank (NULL).

- (4) course_id -> instructor_id -> instructor_office. (The office depends on the instructor, which depends on the course).

- (5) Student (student_id, student_name)

Justification: Separated due to student_id -> student_name.

Instructor (instructor_id, instructor_name, instructor_office)

Justification: Separated due to instructor_id -> instructor_name, instructor_office.

Course (course_id, course_name, instructor_id)

Justification: Separated due to course_id -> course_name, instructor_id.

Enrollment (student_id, course_id, grade, enrollment_date)

Justification: Separated due to student_id, course_id -> grade, enrollment_date.

PART-IV

- (a) I chose a LEFT JOIN ... IS NULL approach over a NOT EXISTS subquery. I made this choice primarily for readability and semantic flow. The LEFT JOIN explicitly lays out the relationship between the tables first, and then cleanly filters the "misses" at the end. While NOT EXISTS is highly performant in modern engines and prevents duplicate row complications, the anti-join pattern (IS NULL) is often easier to debug visually.
- (b) Normalizing the Enrollment table into 3NF eliminates data redundancy and prevents update anomalies, but it heavily impacts query convenience and read performance. Once split, retrieving a simple class roster with student names, course names, and the instructor's office requires joining four distinct tables. Real systems (like Data Warehouses) often intentionally retain denormalization (storing data in 1NF or 2NF) to optimize read speeds and avoid the computational overhead of complex, multi-table joins during analytical queries.